

Proyecto de Software

Entornos, dependencias y reproducibilidad

Tema

- Entornos de ejecución.
- Dependencias directas y transitivas.
- Reproducibilidad.
- Declaración y resolución de dependencias.
- Poetry como herramienta de la cátedra.
- Configuración y secretos.

El problema

"En mi computadora funciona"

Una aplicación depende de mucho más que su código fuente:

- versión de Python;
- librerías y sus versiones;
- configuración;
- eventualmente, servicios externos.

¿Cómo hacemos para que el proyecto se ejecute en otra computadora, en un servidor o dentro de varios meses?

Un mismo equipo, varios proyectos

Supongamos:

Proyecto A

- Python 3.x
- Flask versión X
- SQLAlchemy versión Y

Proyecto B

- Python 3.x
- Flask versión Z
- SQLAlchemy versión W

¿Qué pasa si las versiones requeridas son incompatibles?

Entorno aislado

Un entorno virtual permite disponer de un ambiente aislado para un proyecto.

Nos ayuda a:

- aislar dependencias;
- trabajar con distintas versiones de paquetes;
- evitar modificar el entorno global;
- reconstruir el ambiente de desarrollo.

Cada proyecto trabaja con su propio conjunto de paquetes.

venv: el mecanismo estándar

Python incluye soporte para crear entornos virtuales:

```
python3 -m venv .venv
```

Activación en Linux/macOS:

```
source .venv/bin/activate
```

`venv` resuelve el aislamiento. Todavía necesitamos describir qué instalar.

Pero aislar no alcanza

Crear un entorno vacío no nos dice:

- qué dependencias necesita el proyecto;
- cuáles elegimos directamente;
- qué otras dependencias aparecen;
- qué versiones concretas deben instalarse;
- cuáles son solo para desarrollo.

Necesitamos también **declarar y resolver las dependencias**.

Dependencias directas y transitivas

- **Directa:** la elegimos explícitamente para nuestro proyecto.
- **Transitiva:** llega porque otra dependencia la necesita.

```
mi aplicación
├── Flask
│   ├── Werkzeug
│   ├── Jinja
│   └── ...
```

directa
transitiva
transitiva

Instalar una sola biblioteca puede incorporar varias dependencias adicionales.

Reproducibilidad

```
repositorio
+
definición del proyecto
+
dependencias resueltas
+
configuración documentada
↓
entorno reproducible
```

El objetivo no es **copiar** un entorno virtual, sino poder **reconstruirlo**.

Declarar no es resolver

Declaración

Expresa qué necesita el proyecto y qué versiones son compatibles.

```
flask >= 3.0, < 4.0
```

Resolución

Selecciona versiones concretas compatibles para todo el árbol.

```
Flask 3.x + Werkzeug A + Jinja B + ...
```

¿Qué debemos versionar?

Generalmente sí

- código fuente y definición del proyecto;
- declaración de dependencias y lock file;
- documentación y configuración necesaria, sin secretos.

Generalmente no

- `.venv`;
- cachés y artefactos generados;
- contraseñas, claves y otros secretos.

Git guarda la **receta para reconstruir** el entorno, no el entorno completo.

Antecedente: pip y requirements.txt

`pip` y `requirements.txt` siguen siendo una alternativa válida:

```
pip install flask  
pip freeze > requirements.txt  
pip install -r requirements.txt
```

`pip freeze` refleja principalmente:

qué quedó instalado

Incluye dependencias directas y transitivas sin distinguir por qué están allí.

Dos preguntas diferentes

Una definición moderna del proyecto permite distinguir:

| ¿Qué dependencias declaramos directamente?

de:

| ¿Qué versiones concretas resultaron de resolver el árbol?

Esta separación facilita revisar cambios y reconstruir el entorno.

En la cátedra: Poetry

Durante el proyecto vamos a utilizar **Poetry**.

Poetry gestiona:

- dependencias y sus versiones;
- entorno virtual;
- resolución del árbol de dependencias;
- archivo de bloqueo.

Archivos importantes: `pyproject.toml` y `poetry.lock`.

La herramienta concreta es Poetry. El objetivo sigue siendo un proyecto reproducible.

Flujo de trabajo con Poetry

```
# Inicializar un proyecto existente
poetry init

# Agregar dependencias
poetry add flask
poetry add pytest --group dev

# Ejecutar dentro del entorno
poetry run flask --version

# Reconstruir el entorno de un proyecto existente
poetry install
```

`poetry install` es el caso habitual al incorporarnos a un proyecto del equipo.

pyproject.toml y poetry.lock

`pyproject.toml` describe el proyecto y **declara** sus dependencias.

```
[project]
requires-python = ">=3.11"
dependencies = ["flask (>=3.0,<4.0)"]
```

```
pyproject.toml
    ↓ Poetry resuelve el árbol
poetry.lock
```

`poetry.lock` registra la **resolución concreta**: versiones directas y transitivas.

Los dos archivos forman parte del proyecto y deben versionarse.

Producción y desarrollo

No todas las dependencias cumplen el mismo rol.

Producción: Flask, SQLAlchemy.

Desarrollo: pytest, linters, formatters.

```
poetry add pytest --group dev
```

¿Necesitamos en producción todas las herramientas que usamos durante el desarrollo?

Trabajo en equipo

Un integrante agrega una dependencia:

```
poetry add nueva-libreria
```

Poetry puede modificar `pyproject.toml` y `poetry.lock`. Ambos cambios se versionan con Git.

Otro integrante obtiene los cambios y ejecuta:

```
poetry install
```

Las dependencias son parte del proyecto; sus cambios también deben versionarse.

Configuración ≠ código

No deberíamos escribir secretos directamente en el código:

```
PASSWORD = "mi_clave"
```

La aplicación puede leer configuración desde variables de entorno:

```
DB_HOST  
DB_USER  
DB_PASSWORD  
SECRET_KEY
```

La configuración necesaria se documenta. Los valores secretos **no se versionan**.

¿Y si usamos un agente de programación?

Un agente puede agregar dependencias, modificar `pyproject.toml`, actualizar `poetry.lock` y ejecutar comandos.

El equipo debe revisar:

- qué agregó y por qué;
- qué versiones cambiaron;
- si incorporó dependencias innecesarias;
- si modificó configuración o incluyó secretos;
- si el proyecto continúa siendo reproducible.

Automatizar una modificación no transfiere la responsabilidad sobre el proyecto.

Para pensar

Un compañero clona nuestro repositorio.

¿Cuál de estas situaciones sería deseable?

A

"Instalá cosas hasta que funcione."

B

"Copiá mi carpeta `.venv`."

C

"Usá la definición del proyecto para reconstruir el entorno."

Las herramientas cambian; el problema permanece

Existen `venv`, `virtualenv`, Poetry, Conda, uv y otras herramientas.

En la cátedra utilizaremos **Poetry**.

Los conceptos que deben permanecer son:

- aislamiento;
- dependencias explícitas;
- resolución de versiones;
- reproducibilidad;
- configuración controlada.

Lo que sigue

Ahora que podemos definir y reproducir el entorno del proyecto, vamos a utilizarlo para ejecutar nuestra primera aplicación web con **Flask**.

Referencias

- [Python Packaging User Guide](#)
- `venv`
- [PyPI](#)
- [Poetry: uso básico](#)
- [Poetry: grupos de dependencias](#)
- [uv](#)

Fin